12576 - Nisanth Saravanan
L1 React Assessment

Travel Booking System

Case Study: Travel Booking System: This system manages travel bookings, including traveller names, destinations, travel dates, and booking statuses. It streamlines the booking process and enhances travel planning.

Entity: Booking

Required Fields: BookingId, TravelerName, Destination, TravelDate, BookingStatus

GitHub Link:

Nisanth-Saravanan/L1_React_Assessment: L1 React Assessment

W3H Analysis

Project Title: Travel Booking System	
What?	**How?**
1.What are the Modules needed? **A: Modules Required:** **a) Landing Component.** **b) Bookings Component.** **c) Display Functionality.** **d) Search Functionality.** **e) Delete Functionality.** **f) Update Functionality.** **2. What are the requirements of Landing Component?** **A: Landing Component:** **a) Greet the user with a custom message.** **b) Redirect or Render in the Booking Component.** **c) Return back to Landing Component.** **3. What are the requirements of the Bookings Component?** **A: Bookings Component:** **a) Get User Input.** **b) Validate user input.** **c) Inject new data into the JSON file.** **4. What are the requirements of the Display functionality?** **A: Display Functionality:** **a) Display user data.** **b) Provide means to search.** **c) Provide means to update.** **d) Provide means to delete.**	**1. Landing Component:** **a) Greet the user with a custom message:** **Method 1: Use simple container to greet the users in a landing page.** Method 2: Greet users in an alert. Method 3: Have a series of prompts that lead the user to add a booking by default. **b) Redirect or Render in the Booking Component.** **Method 1: Have simple buttons to render in the component.** Method 2: Have a dedicated Navigation Bar. Method 3: Have the user be automatically redirected to the component. **c) Return back to Landing Component.** **Method 1: Have a simple button to re-render the landing page alone.** Method 2: Have a dedicated Navigation Bar. Method 3: Have the user refresh the page to revisit the landing page. **2. Bookings Components:** **a) Get User Input:** **Method 1: Have a simple form.** Method 2: Use a series of prompts. Method 3: Use modular forms with optional modules. **b) Validate User Input:** **Method 1: User JS validations and alerts.** Method 2: Use default validations.

5. What are the requirements of the Search functionality? **A: Search Functionality:** **a) To search the bookings based on Booking ID.** **b) To display search results.** **6. What are the requirements of the Delete Functionality?** **A: Delete Functionality:** **a) To delete the booking selected.** **7. What are the requirements of the Update Functionality?** **A: Update Functionality:** **a) To update the booking selected.** **b) To validate the update.**	Method 3: Use JS validations and HTML elements. **c) Inject new data into JSON file:** **Method 1: Use Sync Axios.** Method 2: Use Redux. Method 3: Use Async Axios. **3. Display Functionality:** **a) Display user data:** **Method 1: Display as table.** Method 2: Display as cards. Method 3: Display as detailed 's **b) Provide means to search:** **Method 1: Use a button and prompts.** Method 2: Use input field and ask id. Method 3: Render a separate component. **c) Provide means to update:** **Method 1: Use a button and prompts.** Method 2: Use input field and ask id. Method 3: Render a separate component. **d) Provide means to delete:** **Method 1: Use a button and prompts.** Method 2: Use input field and ask id. Method 3: Render a separate component. **4.Search Functionality:** **a) To search the bookings based on Booking ID:** **Method 1: Use prompt and ask for unique booking ID.** Method 2: Use Input field and ask for unique booking ID. Method 3: Use range filters. **b) To display search results:** **Method 1: Use prompts.** Method 2: Use cards. Method 3: Re-render the table. **5.Delete Functionality:** **a) To delete the booking selected:** **Method 1: Use a simple button next to a booking.**

	Method 2: Use input field and ask for id. Method 3: Use filters to select a booking. **6. Update Functionality:** **a) To update the booking selected:** **Method 1: Use prompts to get user input.** Method 2: Use form's input field. Method 3: Render a separate component for updating. **b) To validate the update:** **Method 1: User JS validations and alerts.** Method 2: Render components to showcase validations. Method 3: Use JS validations and HTML elements.
1. Landing Component: **a) Greet the user with a custom message:** **Method 1: Use simple container to greet the users in a landing page.** A: Having to use a simple landing page give a soft welcome to our app. **b) Redirect or Render in the Booking Component.** **Method 1: Have simple buttons to render in the component.** A: Gives an easy approach for accessing the needed component. **c) Return back to Landing Component.** **Method 1: Have a simple button to re-render the landing page alone.** A: Gives an easy approach for accessing the needed component. **2. Bookings Components:** **a) Get User Input:** **Method 1: Have a simple form.**	**1. Landing Component:** **a) Greet the user with a custom message:** Greeting users in an alert and having a series of prompts are improper and over the top respectfully. **b) Redirect or Render in the Booking Component.** A nav-bar for a simple booking and displaying app will have more rendering than needed, and to have the auto redirect will affect the UX. **c) Return back to Landing Component.** A nav-bar for a simple booking and displaying app will have more rendering than needed, and to have the user refresh will affect UX. **2. Bookings Components:** **a) Get User Input:** Multiple prompts for every booking will affect the UX and modular forms are an overkill for a simple requirement.

A: Efficient and can be easily handled to our requirements. **b) Validate User Input:** **Method 1: User JS validations and alerts.** A: Gives an efficient, yet effective means of validations. **c) Inject new data into JSON file:** **Method 1: Use Sync Axios.** A: Gives enough resources to have a smoothly operating data flow in given tie. **3. Display Functionality:** **a) Display user data:** **Method 1: Display as table.** A: Provides a clean UI that can be easily manipulated to our requirements. **b) Provide means to search:** **Method 1: Use a button and prompts.** A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality. **c) Provide means to update:** **Method 1: Use a button and prompts.** A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality. **c) Provide means to update:** **Method 1: Use a button and prompts.** A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality. **4.Search Functionality:** **a) To search the bookings based on Booking ID:**	**b) Validate User Input:** Default validations can not state out need and using HTML elements will increase on screen elements but can be implemented if project gains complexity. **c) Inject new data into JSON file:** Redux has a complex state management and is not needed here, and async axios, even though it is a better option, is not favourable due to time constraints. **3. Display Functionality:** **a) Display user data:** Card-like and separate division add in a lot of spaces in out UI. **b) Provide means to search:** Since the booking id is unique, a button can work effectively and have a very lightweight code behind its working. **c) Provide means to update:** Since the booking id is unique, a button can work effectively and have a very lightweight code behind its working. **c) Provide means to update:** Since the booking id is unique, a button can work effectively and have a very lightweight code behind its working. **4.Search Functionality:** **a) To search the bookings based on Booking ID:** Since the booking id is unique, a prompt can work effectively and have a very lightweight code behind its working. **b) To display search results:** Since the booking id is unique, only a

Method 1: Use prompt and ask for unique booking ID. A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality. **b) To display search results:** **Method 1: Use prompts.** A: Since the booking id is unique, only a single booking will be returned, and so a prompt will be sufficient. **5.Delete Functionality:** **a) To delete the booking selected:** **Method 1: Use a simple button next to a booking.** A: Provides a clean UI that can be easily manipulated to our requirements, and still have the needed functionality. **6. Update Functionality:** **a) To update the booking selected:** **Method 1: Use prompts to get user input.** A: Effective way to get new data and perfoem validations. **b) To validate the update:** **Method 1: User JS validations and alerts.** A: Gives an efficient, yet effective means of validations.	single booking will be returned, and so a prompt will be sufficient. **5.Delete Functionality:** **a) To delete the booking selected:** Since the booking id is unique, a button can work effectively and have a very lightweight code behind its working. **6. Update Functionality:** **a) To update the booking selected:** A: With a low complexity, using HTML elements for each updates just adds more components, and a separate component is inefficient. **b) To validate the update:** Using HTML elements will increase on screen elements but can be implemented if project gains complexity and rendering components is an overkill.
Why?	**Why Not?**

Code:

App.js

```
import "./App.css";
import React, { useEffect } from "react";
import vid from "./media/Background2.mp4";
import Bookings from "./components/Bookings";
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <div className="App">
        <video autoPlay muted loop className="background" src={vid} />
        <header className="App-header">
          <div className="form-container">
            <h1>Travel System</h1>
            <p>
              Welcome to te Travel System, your one-stop destination for all
              things travel. Whether you're planning a vacation, a business
              trip, or a family getaway, we've got you covered. Our
              user-friendly interface makes it easy to search for destinations,
              book flights, and manage your travel plans. With our secure and
              reliable platform, you can rest assured that your travel
              experience is safe and hassle-free.
            </p>
            <table>
              <tr>
                <td>
                  <Link to="/">
                    <button className="search-button" onClick={() => {document.title = "Z
Travel";}}>Home</button>
                  </Link>
                </td>
                <td>
                  <Link to="/bookings">
                    <button className="search-button">Bookings</button>
                  </Link>
                </td>
              </tr>
            </table>
            <Routes>
              <Route path="/" element={<></>} />
              <Route path="/bookings" element={<Bookings />} />
            </Routes>
          </div>
        </header>
      </div>
    </BrowserRouter>
  );
}

export default App;
```

Bookings.js

```
import React from "react";
import { useState, useEffect } from "react";
import axios from "axios";
import vid from "../media/BackgroundVideo.mp4";
import "../App.css";

const API_URL = "http://localhost:5000/bookings";

function Bookings() {
  const [bookings, setBookings] = useState([]);

  useEffect(() => {
    document.title = "Z Bookings";
    axios
      .get(API_URL)
      .then((response) => setBookings(response.data))
      .catch((error) => console.log(error));
  }, []);

  const handleAdd = (e) => {
    e.preventDefault();
    const formData = new FormData(e.target);
    const data = {
      bookingid: formData.get("bookingid"),
      name: formData.get("name"),
      destination: formData.get("destination"),
      status: formData.get("status"),
      date: formData.get("date"),
    };
    if (
      !data.bookingid ||
      !data.name ||
      !data.destination ||
      !data.status ||
      !data.date
    ) {
      alert("Please fill in all fields");
    } else if (
      bookings.some((booking) => booking.bookingid === data.bookingid)
    ) {
      alert("Booking ID already exists");
    } else if (data.date < toString(new Date())) {
      alert("Date cannot be in the past");
    } else {
      axios
        .post(API_URL, data)
        .then((response) => setBookings([...bookings, response.data]))
        .catch((error) => console.log(error));
      alert("Booking added successfully");
      e.target.reset();
    }
```

```
};

const handleDelete = (id) => {
  axios
    .delete(`${API_URL}/${id}`)
    .then(() => setBookings(bookings.filter((booking) => booking.id !== id)))
    .catch((error) => console.log(error));
};

const handleUpdate = (id) => {
  let bookingid, name, destination, status, date;

  const dateRegex = /^\d{4}-\d{2}-\d{2}$/;

  while (!bookingid || !name || !destination || !status) {
    bookingid = prompt("Enter new booking ID:");
    if (bookings.some((booking) => booking.bookingid === bookingid)) {
      alert("Update: Booking ID already exists");
      continue;
    }

    name = prompt("Enter new traveller name:");
    destination = prompt("Enter new destination:");

    status = prompt(
      "Enter new booking status: (Pending, Confirmed, Cancelled)"
    );
    status = status.toLowerCase();
    if (
      status !== "pending" &&
      status !== "confirmed" &&
      status !== "cancelled"
    ) {
      alert("Update: Please enter a valid booking status");
      continue;
    }

    date = prompt("Enter new date: YYYY-MM-DD");
    if (!dateRegex.test(date)) {
      alert("Update: Please enter a valid date in the format YYYY-MM-DD");
      continue;
    }
    if (date < toString(new Date())) {
      alert("Update: Date cannot be in the past");
      continue;
    }

    date = new Date(date);

    if (!bookingid || !name || !destination || !status || !date) {
      alert("Update: Please fill in all fields");
    } else {
      name = name.charAt(0).toUpperCase() + name.slice(1);
      status = status.charAt(0).toUpperCase() + status.slice(1);
```

```
      destination =
        destination.charAt(0).toUpperCase() + destination.slice(1);
      axios
        .put(`${API_URL}/${id}`, {
          bookingid: bookingid,
          name: name,
          destination: destination,
          status: status,
          date: date,
        })
        .then(() =>
          setBookings(
            bookings.map((booking) =>
              booking.id === id
                ? {
                    ...booking,
                    bookingid: bookingid,
                    name: name,
                    destination: destination,
                    status: status,
                    date: date,
                  }
                : booking
            )
          )
        )
        .catch((error) => console.log(error));
    }
  }
};

const toString = (date) => {
  date = new Date(date);
  const year = date.getFullYear();
  const month = (date.getMonth() + 1).toString().padStart(2, "0");
  const day = date.getDate().toString().padStart(2, "0");
  return `${year}-${month}-${day}`;
};

const handleSearch = () => {
  const id = prompt("Enter booking ID:");
  const booking = bookings.find((booking) => booking.bookingid === id);
  if (booking) {
    alert(
      `Booking ID: ${booking.bookingid}\nTraveller Name: ${booking.name}\nDestination: $
{booking.destination}\nBooking Status: ${booking.status}\nDate: ${booking.date}`
    );
  } else {
    alert("Booking not found");
  }
};

function DisplayData() {
  if (bookings.length === 0) {
```

```
  return (
   <div className="App" style={{ marginTop: "100px" }}>
    <header className="App-header">
     <div className="form-container">
      <h1>No Bookings Found</h1>
     </div>
    </header>
   </div>
  );
 } else {
  return (
   <div className="App" style={{ marginTop: "100px" }}>
    <header className="App-header">
     <div className="form-container">
      <div
       style={{ display: "flex", flexDirection: "row", gap: "10px" }}
      >
       <h3>Bookings</h3>{" "}
       <button className="search-button" onClick={handleSearch}>
        Search Booking
       </button>
      </div>
      <table className="dataTable" style={{ marginTop: "25px" }}>
       <tr>
        <th>Booking ID</th>
        <th>Traveller Name</th>
        <th>Destination</th>
        <th>Booking Status</th>
        <th>Date</th>
        <th>Update</th>
        <th>Delete</th>
       </tr>
       {bookings.map((booking) => (
        <tr key={booking.id}>
         <td>{booking.bookingid}</td>
         <td>{booking.name}</td>
         <td>{booking.destination}</td>
         <td>{booking.status}</td>
         <td>{toString(booking.date)}</td>
         <td>
          <button
           className="update-button"
           onClick={() => handleUpdate(booking.id)}
          >
           Update
          </button>
         </td>
         <td>
          <button
           className="delete-button"
           onClick={() => handleDelete(booking.id)}
          >
           Delete
          </button>
```

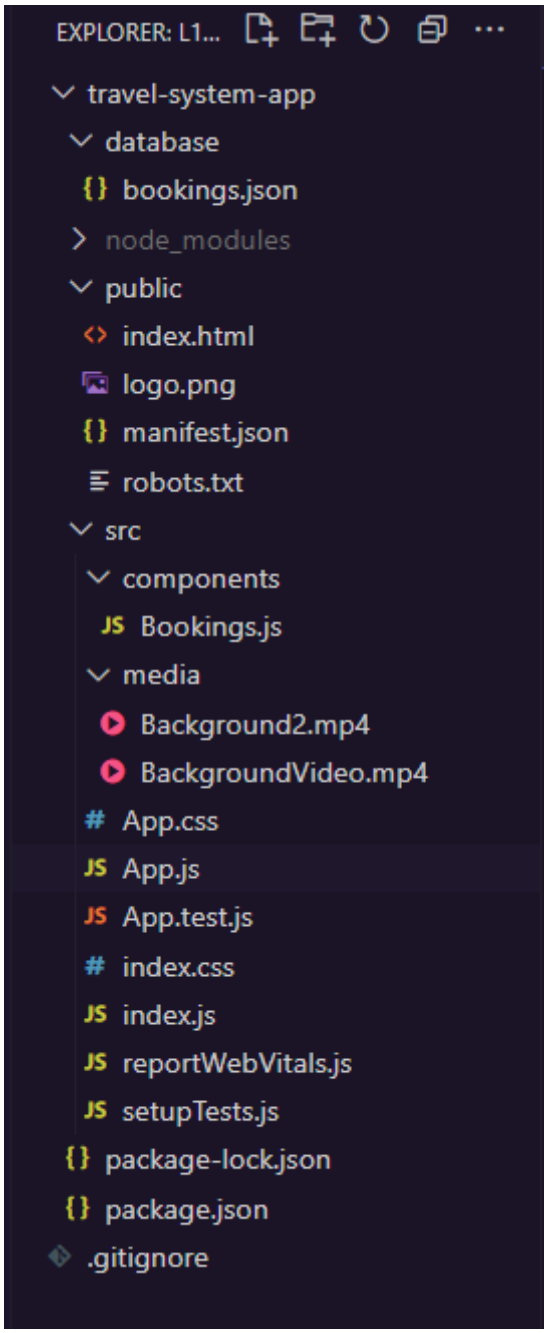
```
            </td>
           </tr>
          ))}
         </table>
        </div>
       </header>
      </div>
    );
  }
}

return (
  <>
    <div className="App">
      <video autoPlay muted loop className="background" src={vid} />
      <header className="App-header">
        <div className="form-container">
          <h3>Bookings Form</h3>
          <form onSubmit={handleAdd} noValidate>
            <table className="formTable">
              <tr>
                <td>
                  <label>Booking ID:</label>
                </td>
                <td>
                  <input
                    type="text"
                    name="bookingid"
                    placeholder="Booking ID"
                  />
                </td>
              </tr>
              <tr>
                <td>
                  <label>Traveller Name:</label>
                </td>
                <td>
                  <input
                    type="text"
                    name="name"
                    placeholder="Traveller Name"
                  />
                </td>
              </tr>
              <tr>
                <td>
                  <label>Destination:</label>
                </td>
                <td>
                  <input
                    type="text"
                    name="destination"
                    placeholder="Destination"
                  />
```

```
            </td>
          </tr>
          <tr>
            <td>
              <label>Booking Status:</label>
            </td>
            <td>
              <select
                name="status"
                placeholder="Booking Status"
                defaultChecked
              >
                <option value="pending">Pending</option>
                <option value="Confirmed">Confirmed</option>
                <option value="Cancelled">Cancelled</option>
              </select>
            </td>
          </tr>
          <tr>
            <td>
              <label>Travel Date:</label>
            </td>
            <td>
              <input type="date" name="date" />
            </td>
          </tr>
          <tr>
            <td></td>
            <td>
              <button type="submit" className="submit-button">
                Submit
              </button>
            </td>
          </tr>
        </table>
      </form>
    </div>
  </header>
</div>

<DisplayData />
</>
);
}

export default Bookings;
```

bookings.json (initial)

```
{
  "bookings": []
}
```

File Structure

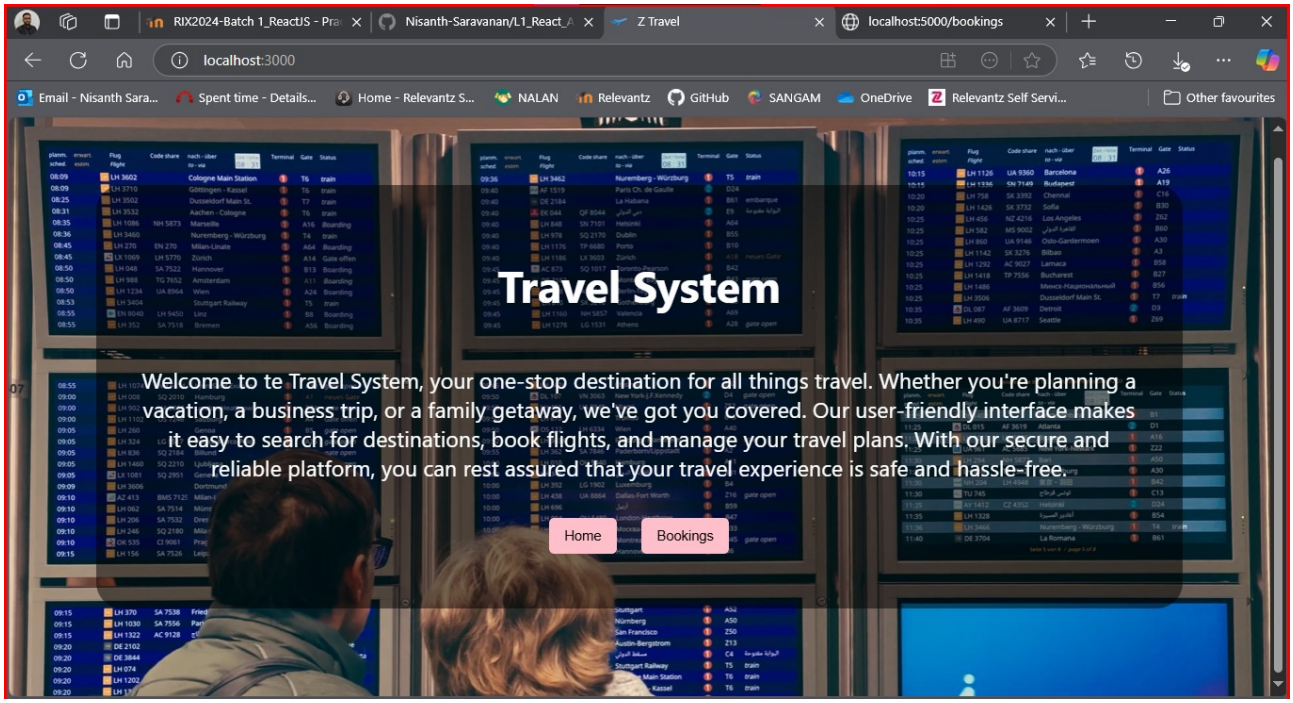


Output Screens:

Key Notes:

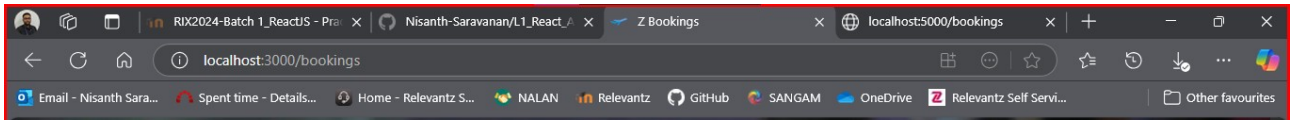
- All Backgrounds are videos and are not static images.
- All fields include validations, including date.
- All bookingid are required to be unique.
- During update, even if the date is given in as a string, it is converted into date-format prior to updation.
- Can not add or update a booking on past date.
- Can not add or update a booking with existing id.
- Can not update a booking with any invalid status.
- The title of the document changes from *“Z Travel”* to *“Z Bookings”* and back.

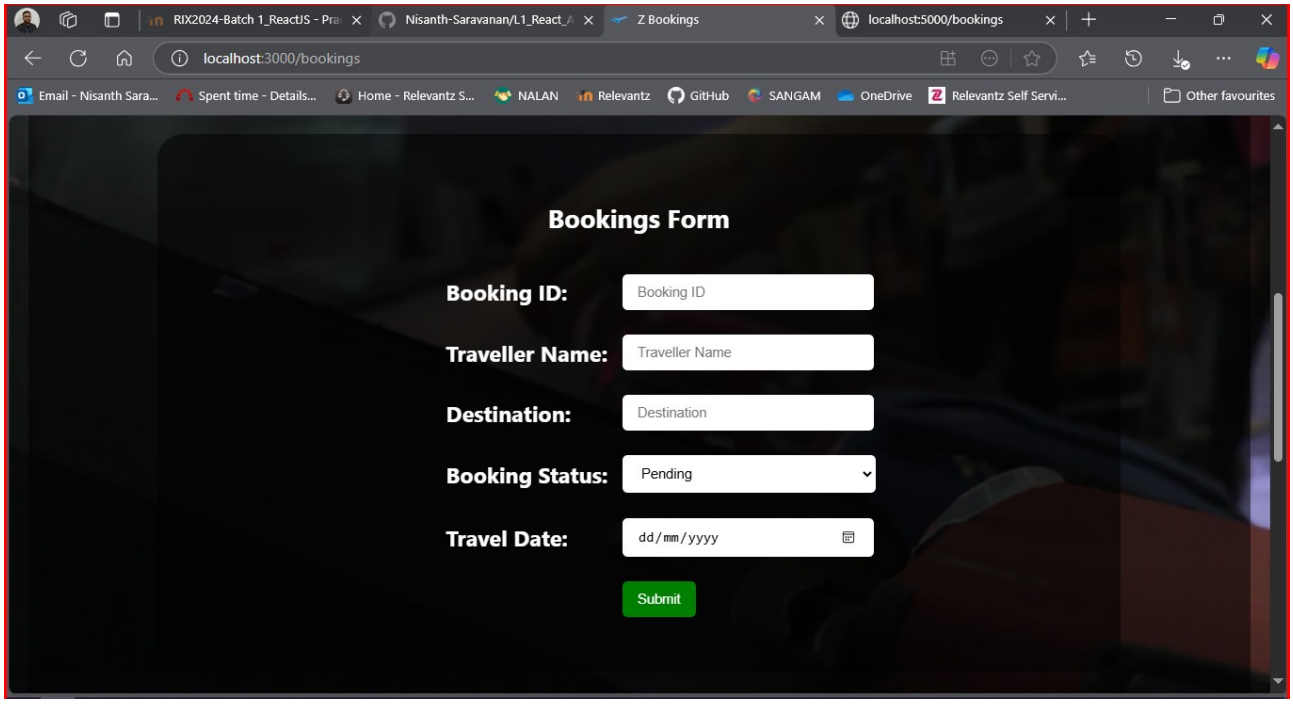
Landing Page: App.js



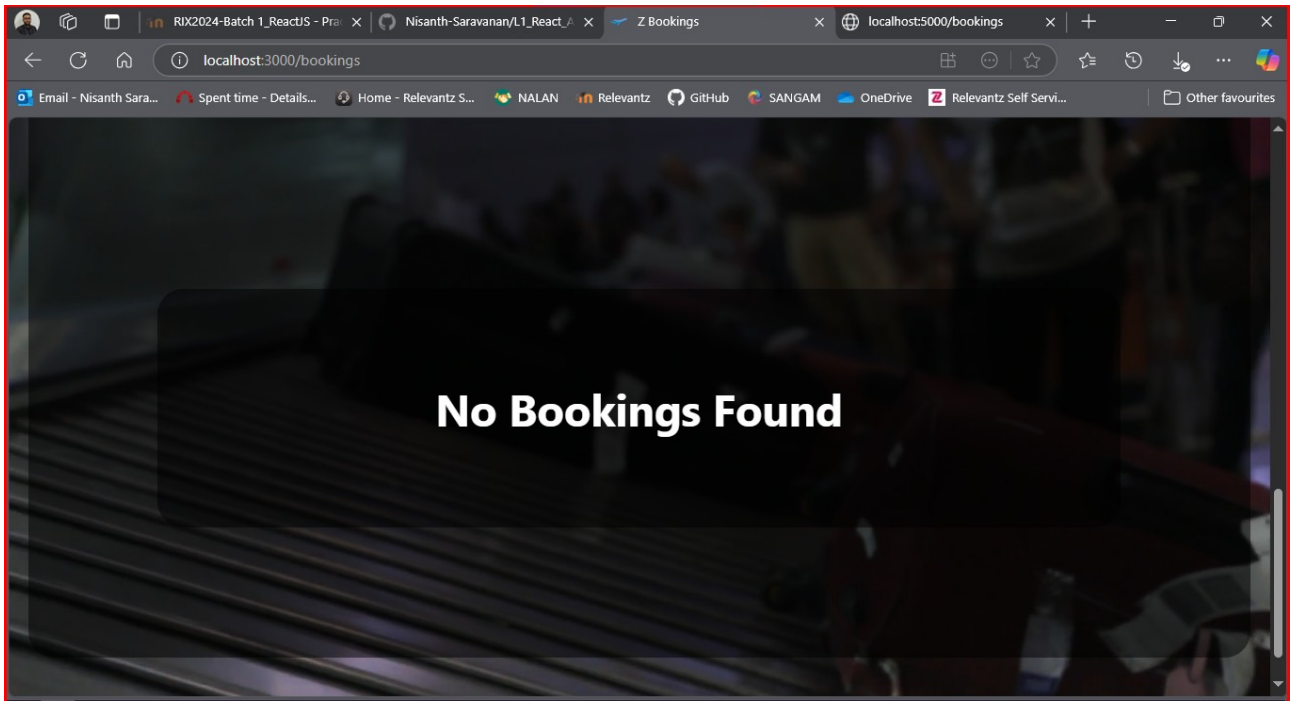
Booking Component:

The bookings component is rendered inside the app.js component, with a diferent background video.



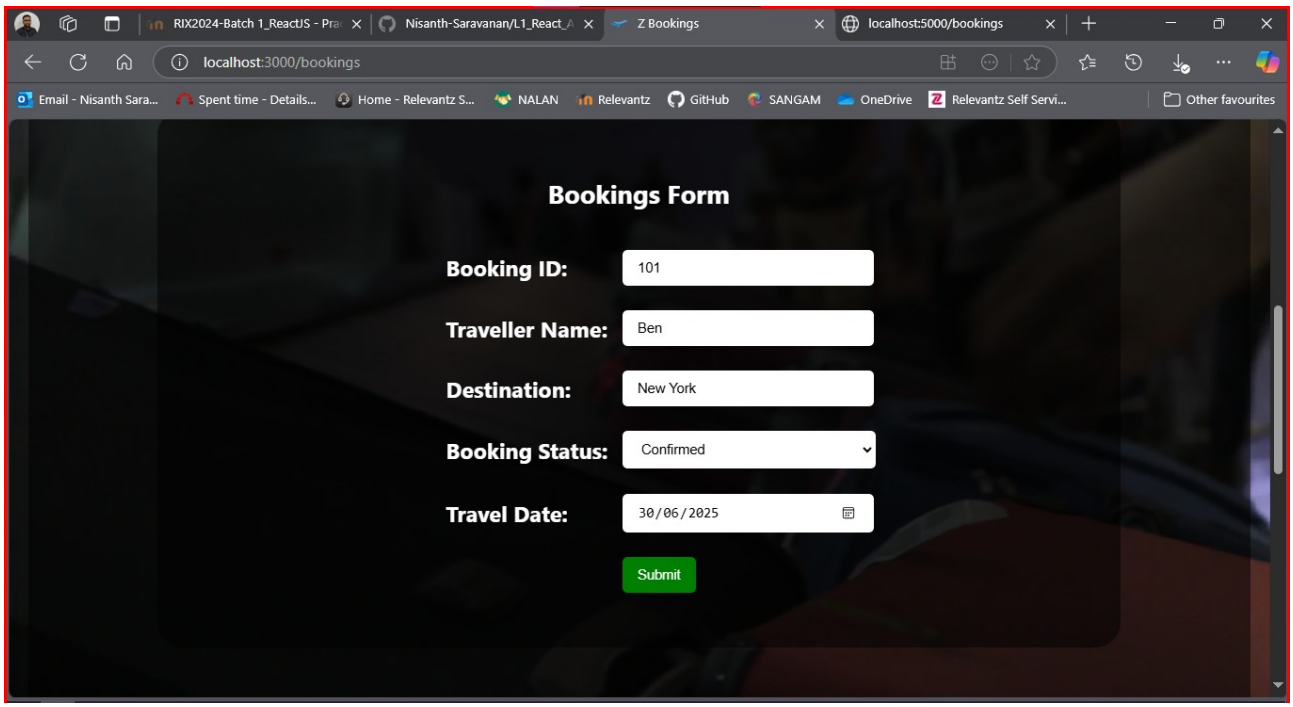


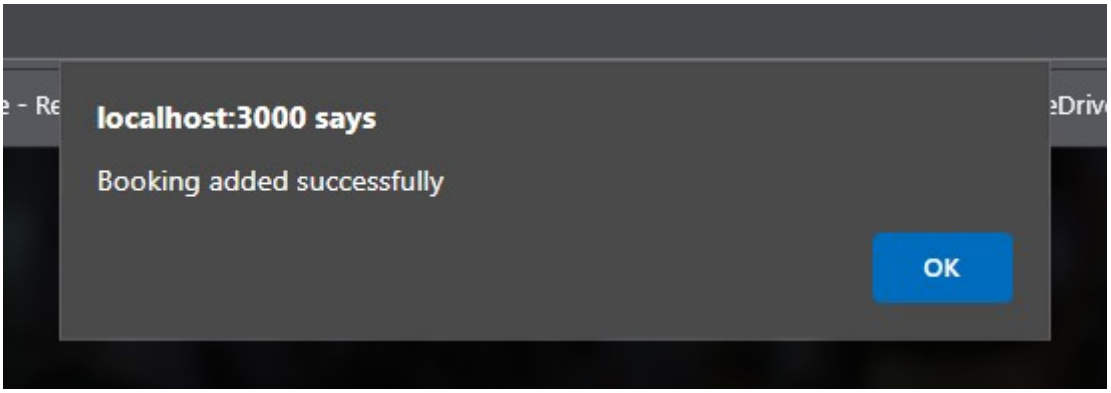
Display (Empty/Initial):



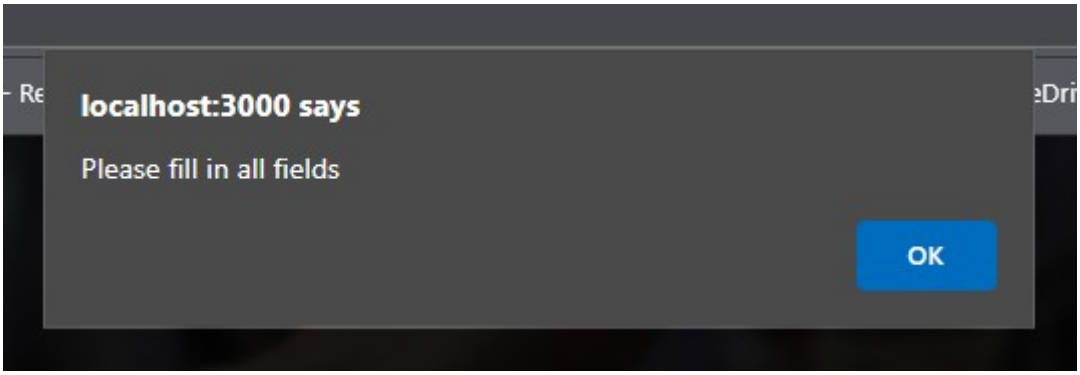
Bookings Form Validations:

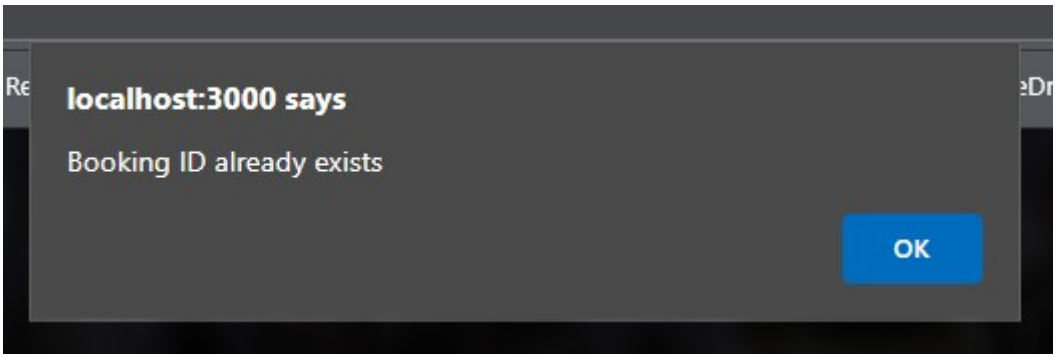
Correct Insert:

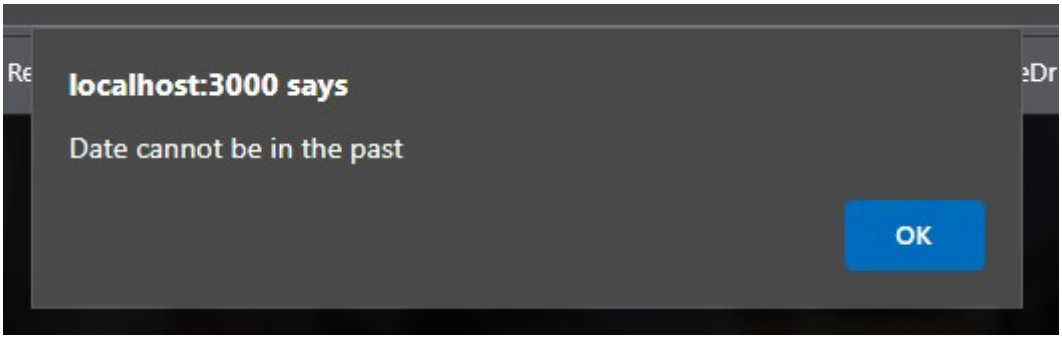




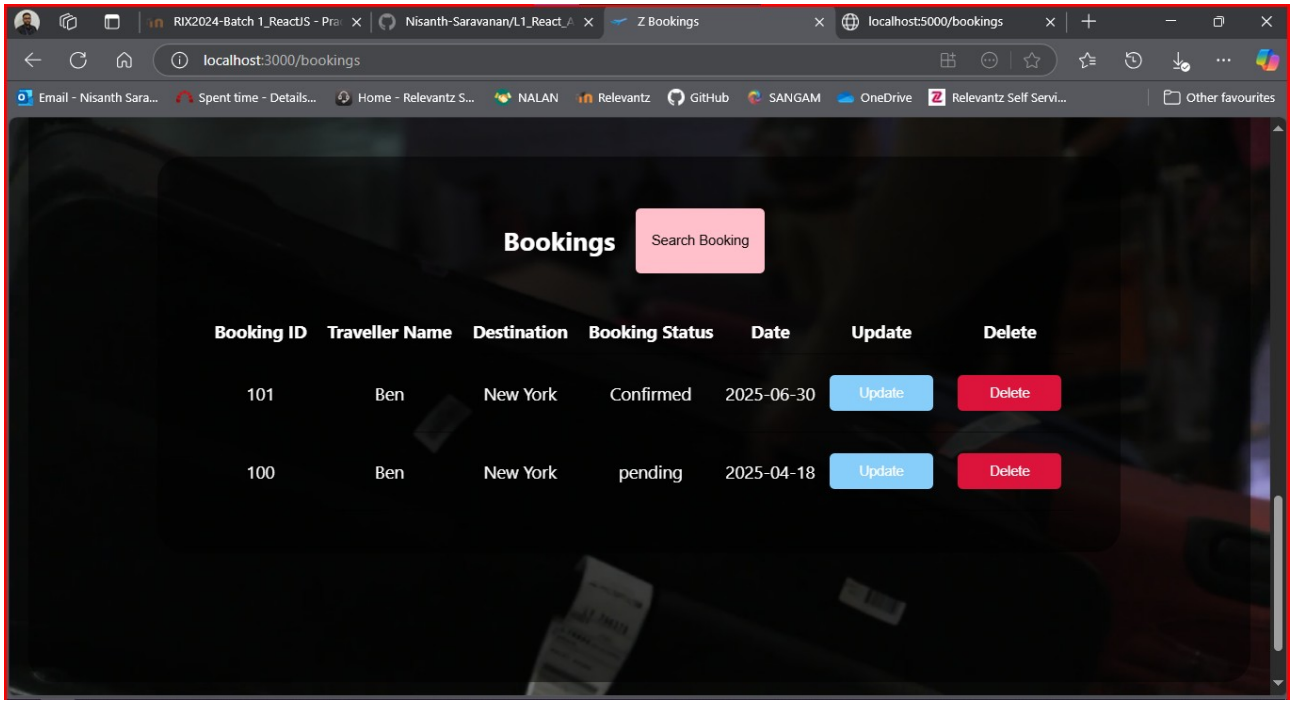
Incorrect Insert:

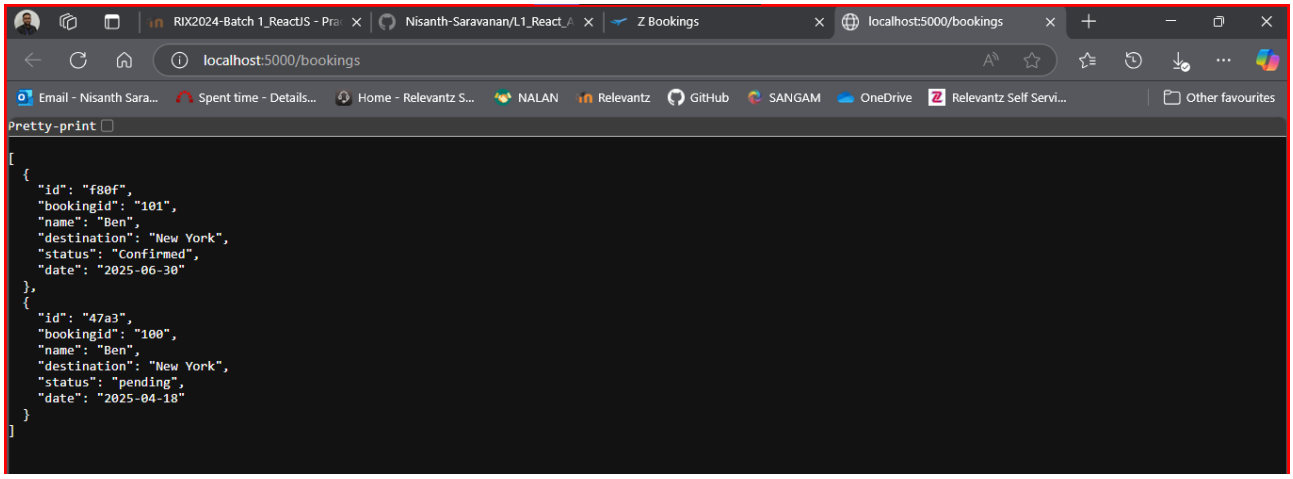




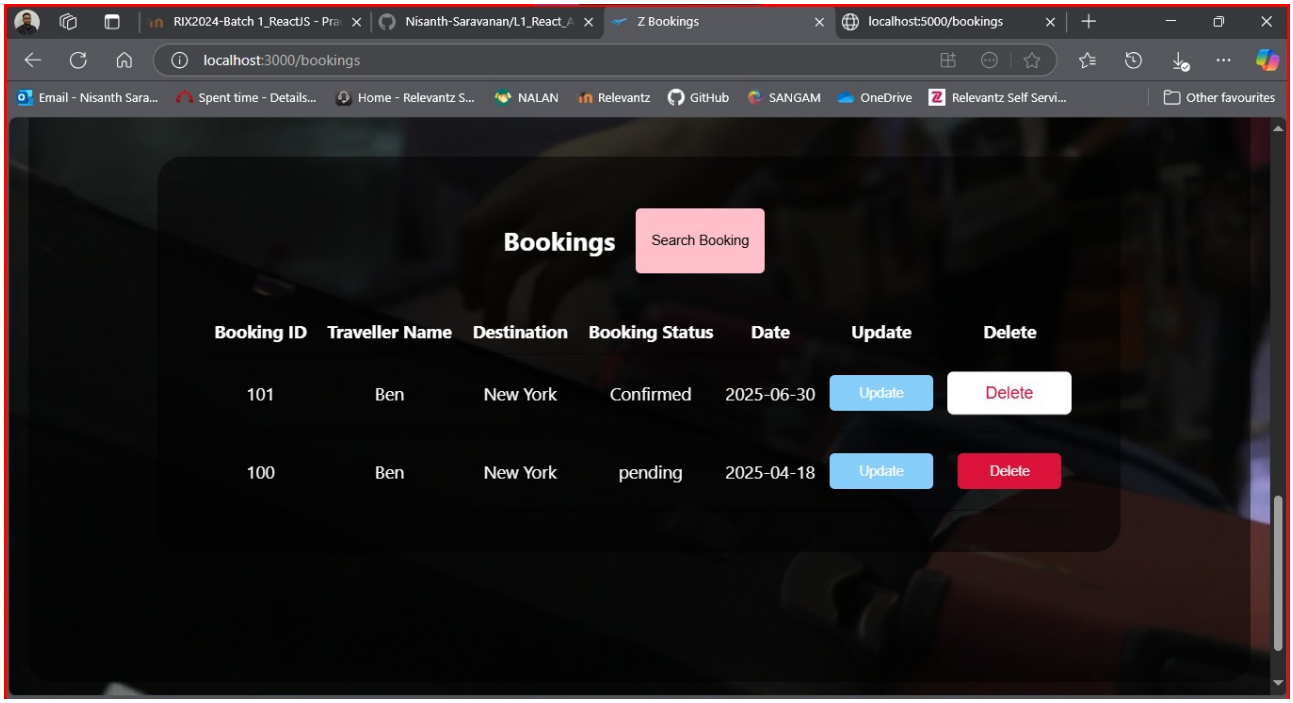


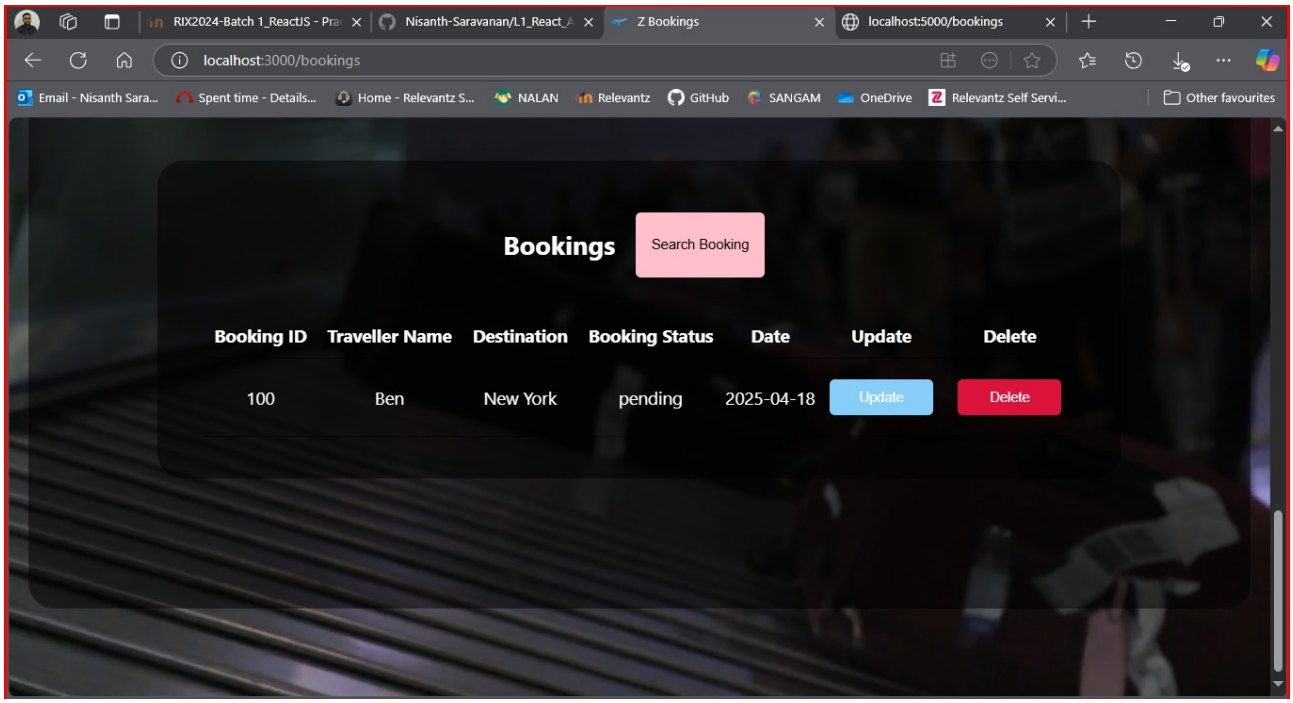
Display (With Data):

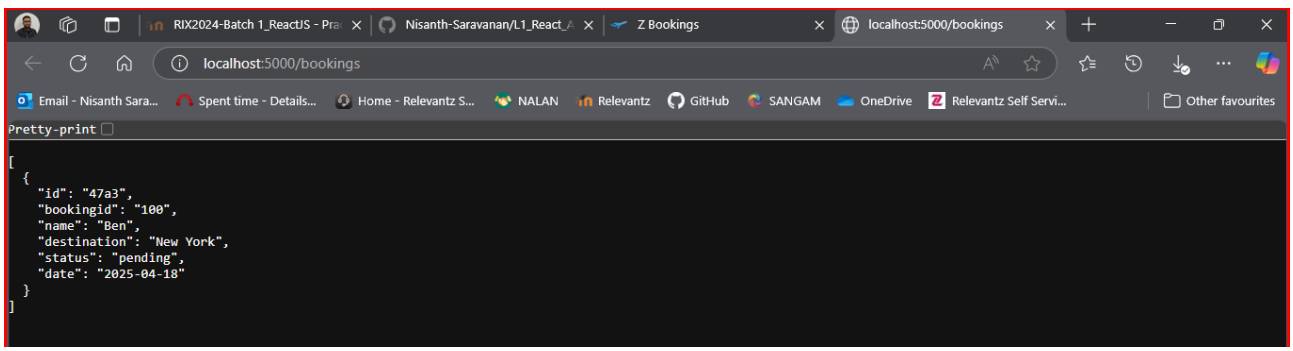




Deletion:

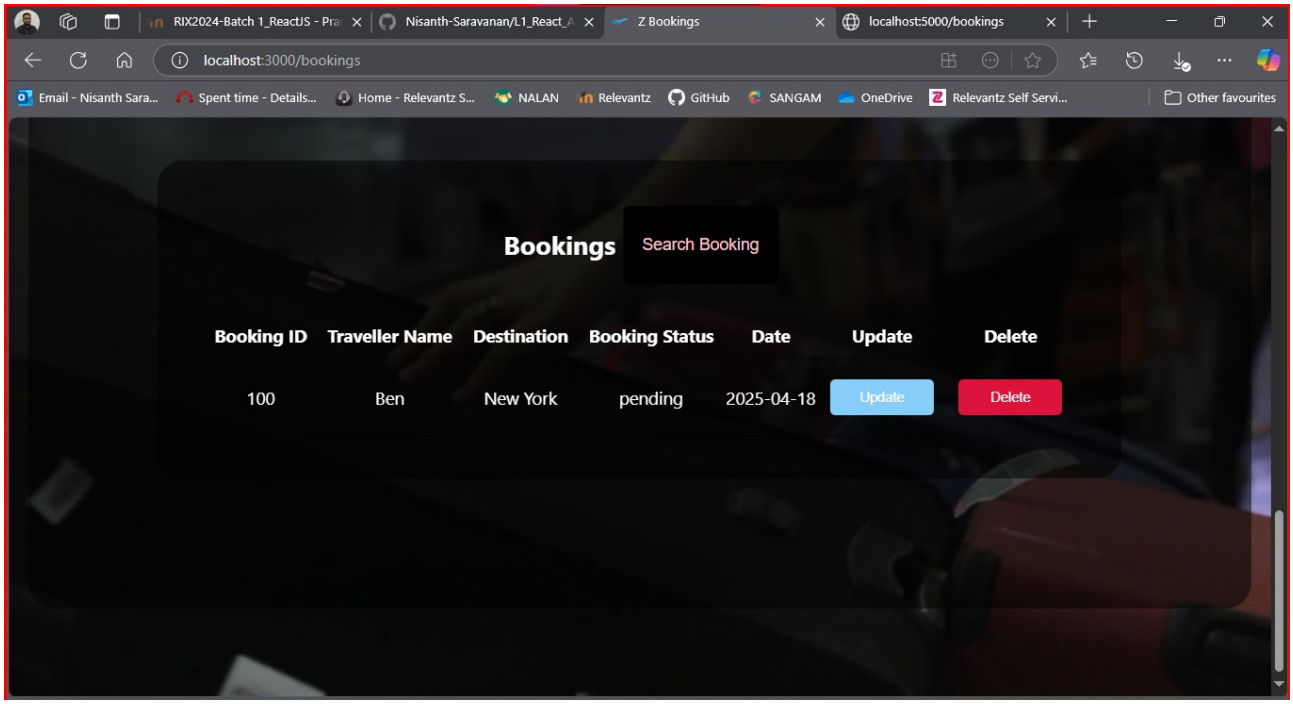

RIX2024-Batch 1_ReactJS - Pra
Nisanth-Saravanan/L1_React_A
Z Bookings
localhost:5000/bookings
localhost:3000/bookings
Email - Nisanth Sara...
Spent time - Details...
Home - Relevantz S...
NALAN
Relevantz
GitHub
SANGAM
OneDrive
Relevantz Self Servi...
Other favourites
Bookings
Search Booking
Booking ID
Traveller Name
Destination
Booking Status
Date
Update
Delete
101
Ben
New York
Confirmed
2025-06-30
Update
Delete
100
Ben
New York
pending
2025-04-18
Update
Delete

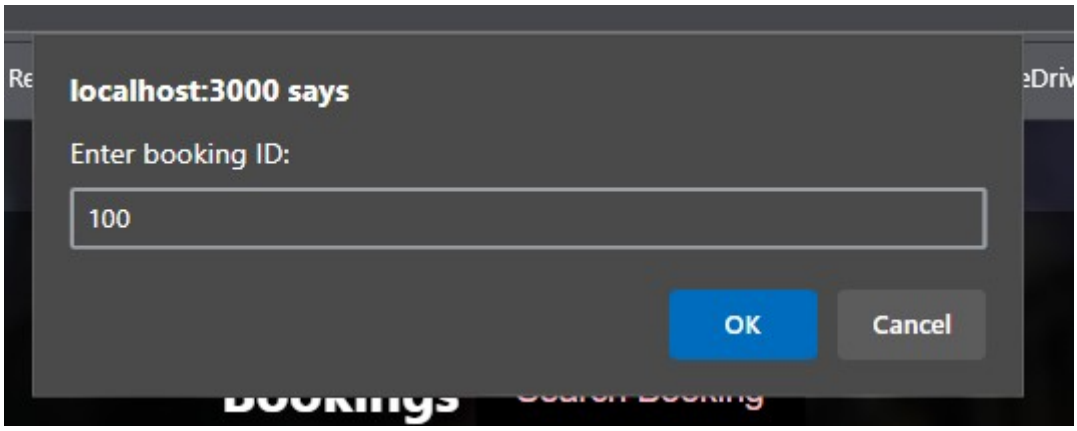

RIX2024-Batch 1_ReactJS - Pra
Nisanth-Saravanan/L1_React_A
Z Bookings
localhost:5000/bookings
localhost:3000/bookings
Email - Nisanth Sara...
Spent time - Details...
Home - Relevantz S...
NALAN
Relevantz
GitHub
SANGAM
OneDrive
Relevantz Self Servi...
Other favourites
Bookings
Search Booking
Booking ID
Traveller Name
Destination
Booking Status
Date
Update
Delete
100
Ben
New York
pending
2025-04-18
Update
Delete

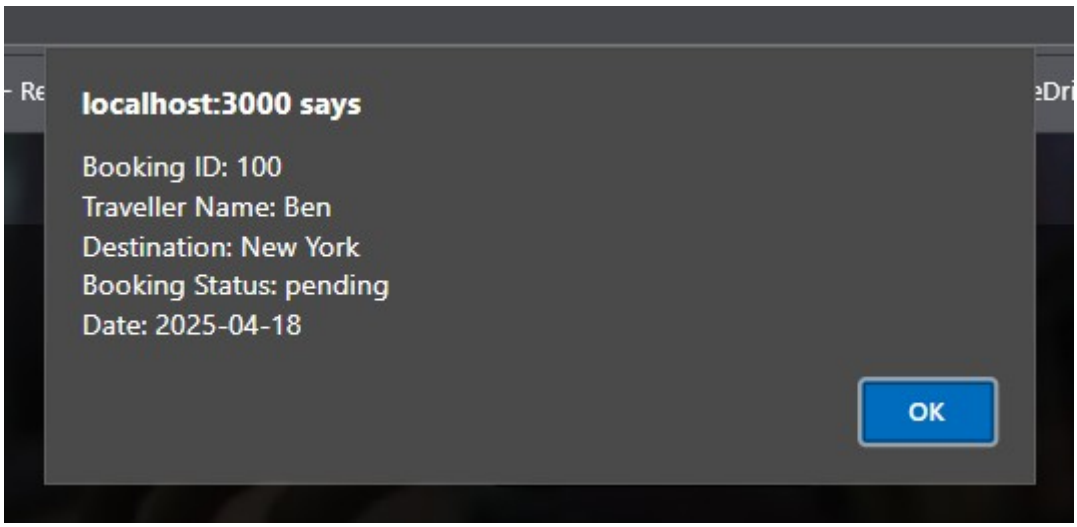

RIX2024-Batch 1_ReactJS - Pra
Nisanth-Saravanan/L1_React_A
Z Bookings
localhost:5000/bookings
localhost:5000/bookings
Email - Nisanth Sara...
Spent time - Details...
Home - Relevantz S...
NALAN
Relevantz
GitHub
SANGAM
OneDrive
Relevantz Self Servi...
Other favourites
Pretty-print
[
{
"id": "47a3",
"bookingid": "100",
"name": "Ben",
"destination": "New York",
"status": "pending",
"date": "2025-04-18"
}
]

Searching:

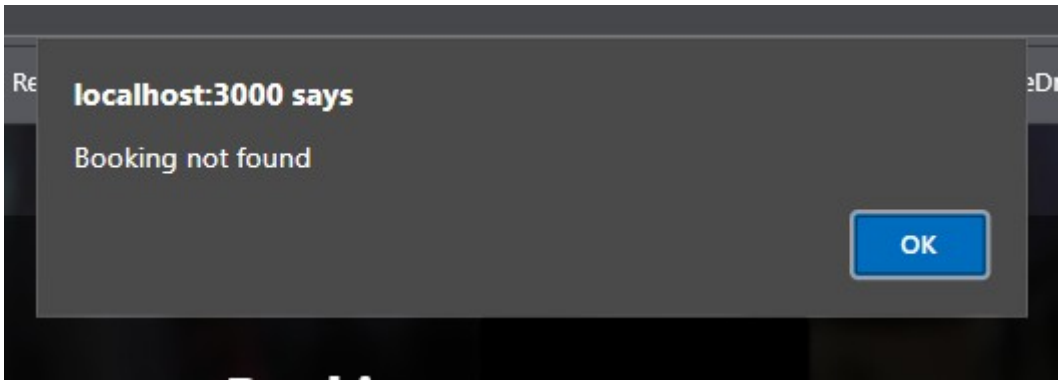
Valid ID:



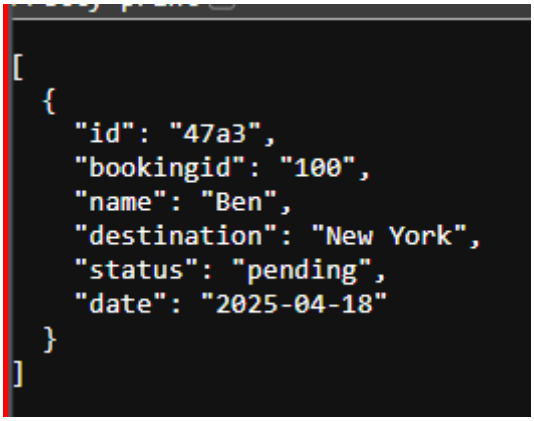



localhost:3000 says
Booking ID: 100
Traveller Name: Ben
Destination: New York
Booking Status: pending
Date: 2025-04-18
OK

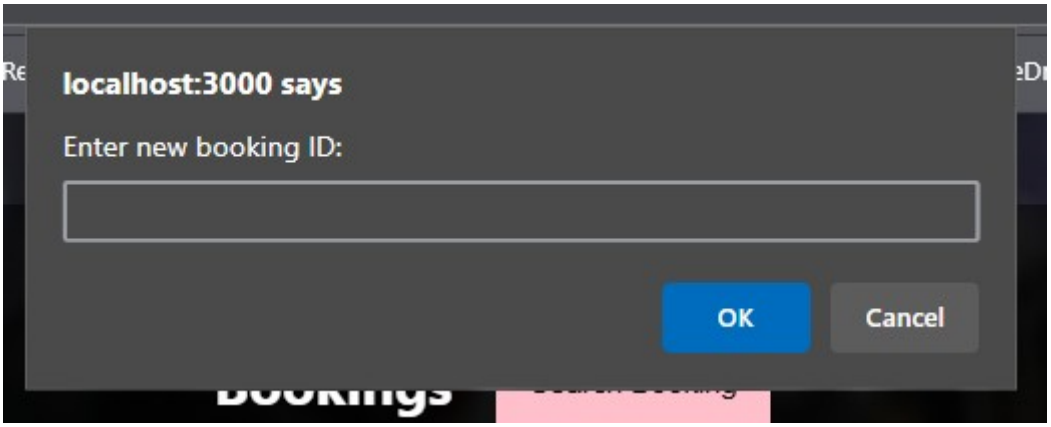
Invalid ID:

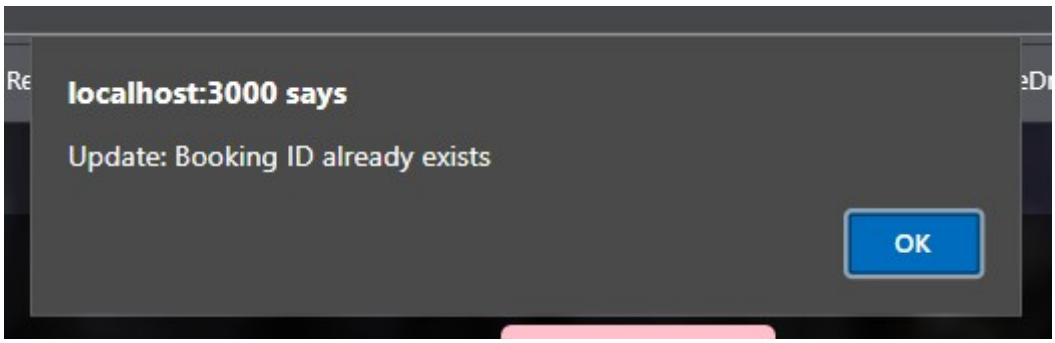

localhost:3000 says
Booking not found
OK

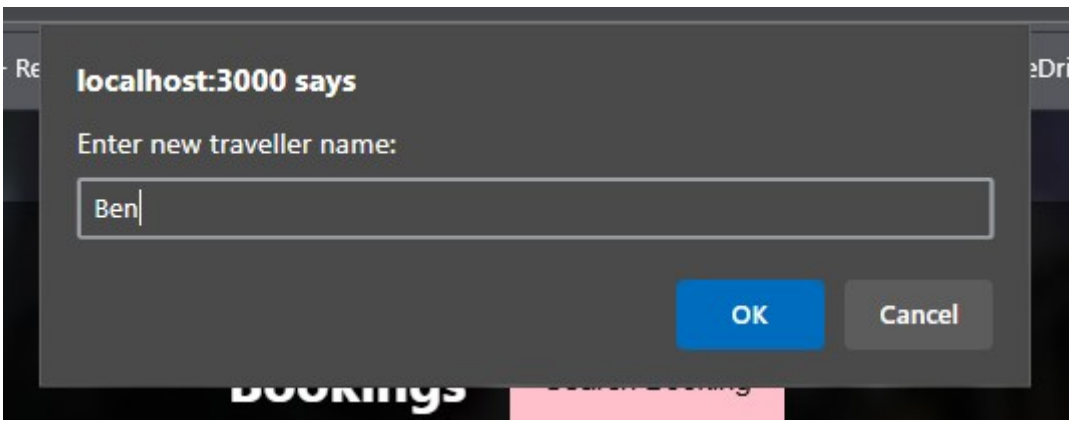
Updating:

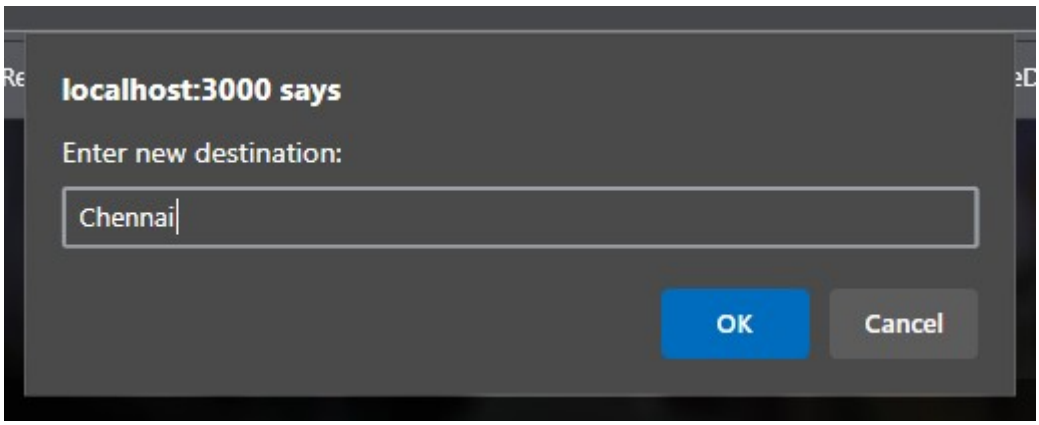


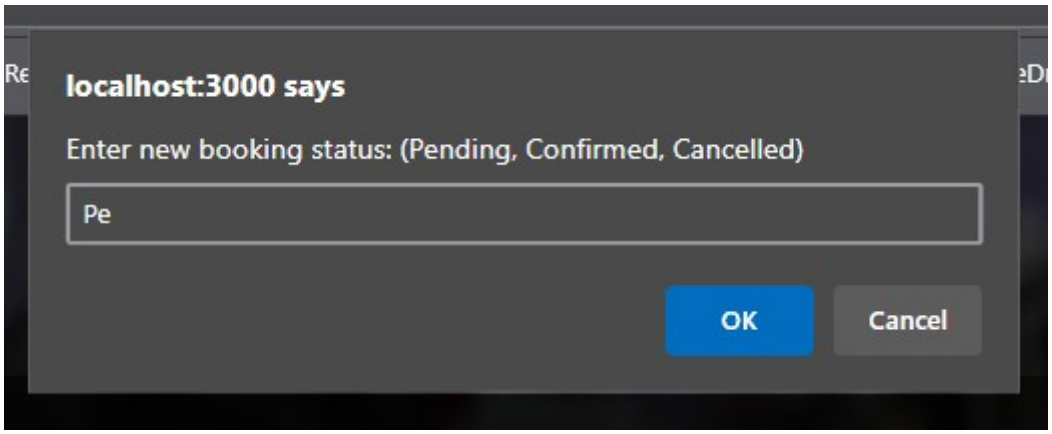
```
[
  {
    "id": "47a3",
    "bookingid": "100",
    "name": "Ben",
    "destination": "New York",
    "status": "pending",
    "date": "2025-04-18"
  }
]
```

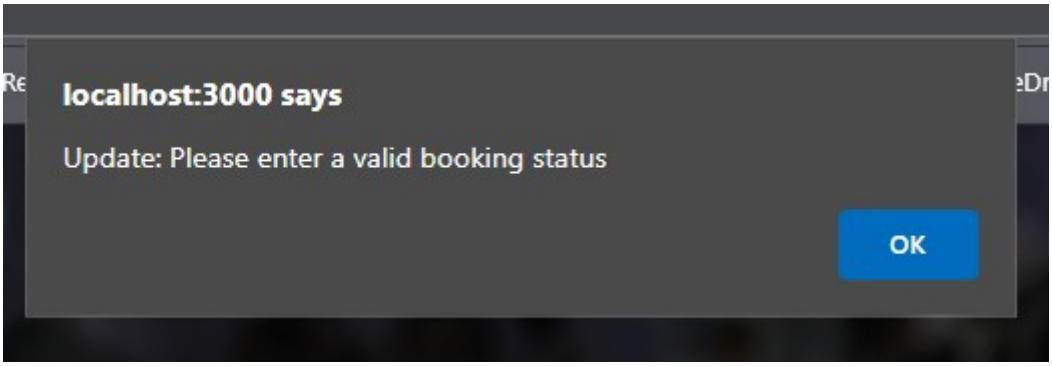


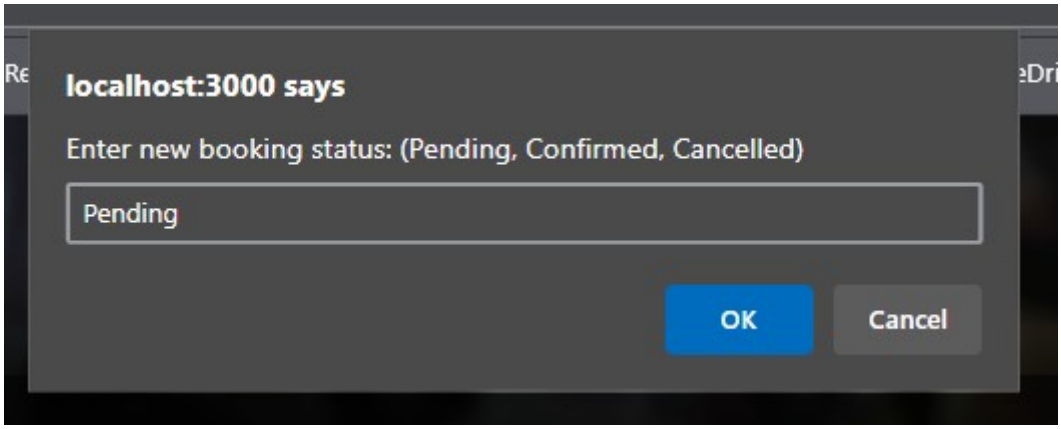


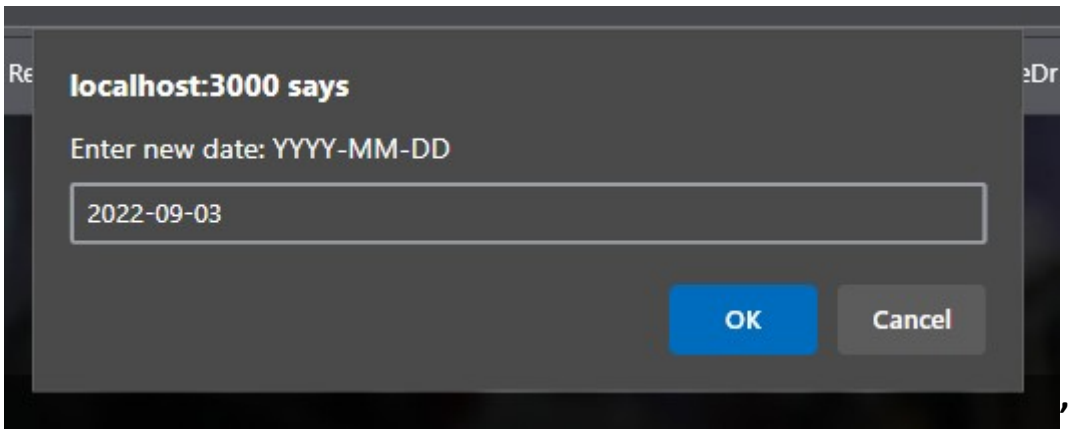


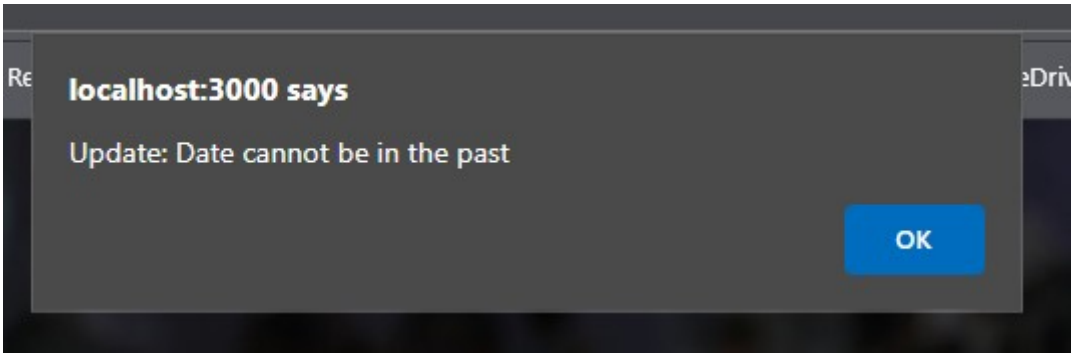

localhost:3000 says
Enter new destination:
Chennai
OK
Cancel


localhost:3000 says
Enter new booking status: (Pending, Confirmed, Cancelled)
Pe
OK
Cancel


localhost:3000 says
Update: Please enter a valid booking status
OK


localhost:3000 says
Enter new booking status: (Pending, Confirmed, Cancelled)
Pending
OK
Cancel





Pretty-print


```
[
  {
    "bookingid": "101",
    "name": "Ben",
    "destination": "Chennai",
    "status": "Pending",
    "date": "2025-09-09T00:00:00.000Z",
    "id": "47a3"
  }
]
```